pass control to its caller using the yield keyword, and RSpec provides four different matchers for specifying this behavior.

yield_control

The simplest yield matcher is yield_control:

11-matchers-included-in-rspec-expectations/03/block_matchers.rb
```
def self.just_yield
  yield
end

expect { |block_checker| just_yield(&block_checker) }.to yield_control
```

In order for the expectation to pass, the just_yield method must yield control to a block, or to an object that acts like a block. RSpec provides just such an object for us: a block checker that verifies that we actually yielded to it. All of the yield matchers use this technique.

You can also specify an expected number of yields by chaining once, twice, thrice, exactly(n).times, at_least(n).times or at_most(n).times.

11-matchers-included-in-rspec-expectations/03/block_matchers.rb
```
expect { |block| 2.times(&block) }.to yield_control.twice
expect { |block| 2.times(&block) }.to yield_control.at_most(4).times
expect { |block| 4.times(&block) }.to yield_control.at_least(3).times
```

The times method is just decoration, but it helps to keep your yield_control expectations readable.

yield_with_args

When you care about which specific arguments your method is yielding, you can check them with the yield_with_args matcher:

11-matchers-included-in-rspec-expectations/03/block_matchers.rb
```
def self.just_yield_these(*args)
  yield(*args)
end

expect { |block|
  just_yield_these(10, 'food', Math::PI, &block)
}.to yield_with_args(10, /foo/, a_value_within(0.1).of(3.14))
```

As this example shows, you can use several different criteria to check a yielded object, including:

- An exact value such as the number 10
- An object implementing ===, such as the regular expression /foo/
- Any RSpec matcher, such as a_value_within()